

S7 Delivery — Sprint Plan

AMS tabletop exercise · S7 (Full-Scale Application Development & Delivery) Demo insurer: MapleSure Insurance (fictional). Written 2026-08-03.

The rule that shapes this plan

No sprint ends without a runnable demo beat.

If a sprint cannot be shown at the end of it, it was scoped wrong. This is not a presentation nicety — it is the scheduling mechanism. A sprint that can only be described is a sprint whose risk is invisible until the week is gone.

Two consequences, applied throughout:

- **Every sprint below names its demo view explicitly.** That view is the exit criterion, not the code.
- **Sprints are ordered so the demo gets more honest, never more impressive.** The run ledger starts by saying "0 of 12 artifacts are AI-generated" and improves from there. At no point does the demo claim more than it has.

Naming — reconciled

The repo previously used *Sprint 0*, *Sprint A*, *Sprint B* and *Sprint 1* for overlapping things. Unified on numbers as of 2026-08-03:

Old	Now	Meaning
Sprint 0 / Sprint A	Sprint 0	Foundation, staged artifacts
Sprint 1	Sprint 1	UserStory contract fixed
Sprint B	Sprint 3	Real LLM calls + committed recordings

Dependency graph



Two ordering constraints are **hard**, not preferences:

1. **Prompt prefix ordering must precede any committed recording.** ✅ **Discharged 2026-08-03** by the Sprint 0 rework. The cache key hashes the prompt by design, so restructuring prefixes after recordings exist invalidates every one of them. This was originally scheduled as Sprint 2; pulling it into Sprint 0 — while the repo had zero LLM callers and zero recordings — made it free. It is now a standing rule rather than a sequencing risk: **prefix order is frozen once Sprint 3 records anything.**
2. **Sprint 1 must precede Sprint 4.** The `UserStory` shape is the interface the downstream consumes. Building the downstream first means guessing at it.

Sprint 4 does not depend on Sprint 3. This is deliberate. LLM access is an open external blocker; routing everything through `s7_delivery/models.py` means the downstream can be built and demoed on staged artifacts and flip to real output later without changing a line above the model layer. If Sprint 3 stalls, Sprint 4 still ships.

Sprint 0 — Foundation · DONE (reworked 2026-08-03)

Goal. A pipeline that runs end to end on staged artifacts, with the determinism *and* cache-economics model correct before anything depends on them.

Demo view. The delivery console runs all five beats: epic intake → assessment with effort-weighted coverage → DFD/ER diagrams → human review gate → story breakdown. The gate genuinely blocks; rejection keeps stories locked. Every artifact carries a visible `STAGED` badge.

Built. `common/llm.py` (5 providers, replay/record/live, loud replay misses), `common/prompt.py`, `common/telemetry.py`, `s7_delivery/models.py`, `pipeline.py`, `staged.py`, `apps/console/`, `crs/EPIC-S7-001.md`, **60 tests** green offline with no API key.

The rework — why it happened here rather than in Sprint 2

The cache-efficient architecture review (2026-08-03) produced two findings that belong to the **foundation layer**, not to a later sprint: prompt assembly and cache telemetry both live in `common/`. Sprint 2 originally carried the prompt work, which was the wrong home for it.

The deciding fact: at the time of the rework the repo had **zero production LLM callers and zero committed recordings**. Changing prompt assembly costs nothing now and invalidates every recording later. Doing it in Sprint 2 would have meant doing it *after* the interface had users. This is hard ordering constraint 1 applied one sprint earlier than written, which is strictly cheaper.

What changed.

- **`common/prompt.py` (new).** `PromptLayers` assembles a prompt in fixed, stability-ordered layers — `rules` → `role` → `memory` → `ref` → `task` — splitting into `system = rules + role` and `prompt = memory + ref + task`. Providers cache a *prefix*, so a volatile segment placed early makes every stable segment after it a miss. Ordering by stability is the entire mechanism. It is plain string assembly; a provider that does no caching is unaffected.
- **Usage in `common/llm.py`.** The provider contract moved from `(text, int, int)` to `(text, Usage)`. Anthropic and Bedrock now report `cache_read_input_tokens` and `cache_creation_input_tokens`; the OpenAI-compatible providers leave them unset because they do not report them. This also removed a pre-existing `usage.prompt_tokens if usage else 0`, which fabricated a zero whenever a provider returned no usage object.
- **Cache counters in `common/telemetry.py`.** `log_call` carries them, `ScenarioSummary` aggregates them, and `cache_efficiency()` returns the read-to-write ratio — or `None`. It returns `None` when nothing reported counters, and `None` when writes are zero, because a ratio over zero writes is undefined rather than infinite.
- **Recordings persist cache counters**, so a replayed run reports the economics of the call that was actually made instead of collapsing to "not measured" — which matters because replay is the mode the demo runs in.

The discipline, stated once. Everywhere in this layer, an unreported number stays `None` and never becomes `0`. **Zero is a measurement; `None` is an admission.** A provider that cannot measure must not be presented as one that measured a total miss. This is the same rule the existing code already applied to cost, extended to cache.

Known limitation. Every artifact is still hand-written, not model output, and labelled as such everywhere it appears. Streaming reports no cache counters: providers surface them only on a final usage object that not every streaming implementation exposes, so they stay unset there by choice.

Sprint 1 — Surfaces and the run ledger

Goal. Two surfaces over one pipeline, and one honest number about the run.

Demo view. The same five beats, now runnable **two ways** — in the browser and in the terminal — from the same pipeline. A persistent ledger strip reads:

Provenance	12 of 12 artifacts STAGED · 0 replayed-AI · 0 live
AI coverage	58% of estimated effort (3 tasks not AI-addressable)
Economics	Not measured — no live calls in this run
Mode	replay

The demo beat *is* the honesty. "Nothing here is AI-generated yet, and the tool says so itself" is a stronger opening than a claim nobody can check.

Builds.

- `run_ledger()` in `pipeline.py` — one function, single source of truth
- `s7_delivery/cli.py` — `s7 epic` / `s7 run` / `s7 gate` / `s7 export`
- Web ledger strip + mode indicator, consuming the same function
- **Fix the `UserStory` contract** — the blocking decision for Sprint 4

Exit criteria. Both surfaces render identical ledger figures. `UserStory` is frozen and documented. Tests cover the ledger through the CLI (text is assertable; DOM is not).

Blocked by. Nothing. No LLM access required.

Sprint 2 — Artifact plane, stage reuse, prompt ordering

Goal. Make stages independently re-runnable, and make the demo survive a mid-run failure.

Demo view. Run the pipeline. Run it again — stages now light up **REUSED** rather than REGENERATED, because their output already exists and validates against the contract. Then kill it mid-run and resume: it picks up where it stopped instead of starting over.

This is the strongest new beat in the plan. It demonstrates determinism visually in one gesture, and it is the demo-recovery mechanism — a beat that dies in front of the room is recoverable rather than fatal.

Builds.

- `s7_delivery/artifacts.py` — deterministic paths, validate against `models.py`
- Early-exit: valid output present → stage skips

Prompt prefix ordering was originally scoped here. It moved into the Sprint 0 rework on 2026-08-03 — it is `common/` foundation code, and doing it while the repo had no LLM callers and no recordings was free. See Sprint 0.

Exit criteria. A second run of an unchanged epic performs no regeneration. An interrupted run resumes.

Blocked by. Sprint 1 (ledger reports the reuse).

Sprint 3 — Real AI output and committed recordings

Goal. Flip provenance from `STAGED` to `REPLAYED_AI` .

Demo view. The ledger changes on its own:

Provenance	3 of 12 artifacts STAGED · 9 replayed-AI · 0 live
Economics	\$X.XX this run · cache reads N, writes M

And the proof that matters: **a fresh clone with zero API keys runs the whole demo offline**, from committed recordings. That is the reliability claim made checkable rather than asserted.

Builds.

- Swap `staged.py` for real `common.llm` calls, stage by stage
- Record and commit replay recordings to `s7_delivery/cache/llm`
- Wire cache read/write token counts into `telemetry.py` and the ledger

Exit criteria. Fresh clone, no `.env`, full run green. Economics reports **our own measured numbers or nothing** — never zero, never an estimate, and never a figure borrowed from any other team's benchmark.

Blocked by. ⚠️ **LLM access — an open external blocker.** Platform-team approval is not settled. Interim options are an approved internal assistant, a local model, or personal keys. If this stalls, skip to Sprint 4; nothing above `models.py` changes when Sprint 3 eventually lands.

Sprint 4 — The downstream lane

Goal. Close the loop from story to release.

Demo view. Take one story from Sprint 0's breakdown and carry it the rest of the way: generated code → failing tests (red) → passing tests (green) → docs → release record. A **second gate** before release, which blocks the same way the design gate does.

The narrative lands here: the app is where humans direct and review, the CLI is where agents do the work, and the two gates are where control sits.

Builds.

- `build → test → docs → release` stages against the frozen `UserStory`
- Release gate in the app; execution in the CLI
- Decide reuse-vs-rebuild of the sibling S3 downstream — **now** answerable, because the interface it consumes is fixed

Exit criteria. One story traverses the full lane. Both gates block. Stage provenance is labelled throughout.

Blocked by. Sprint 1 (`UserStory` contract). **Not** Sprint 3.

Sprint 5 — Enhancement lane and the KPI scorecard

Goal. Show both entry modes converging, and answer the metrics question.

Demo view. A mode selector on entry. **Project mode** starts at an epic and runs the full upstream; **enhancement mode** starts at user stories and drops straight into the downstream. Both converge on the same build/test/docs/release lane, shown side by side. A scorecard reports the delivery KPIs the run can actually evidence.

Builds.

- Mode selector; S3-style enhancement lane (MapleSure retirement eligibility)
- Delivery KPI scorecard: velocity, cycle time, estimation accuracy, defect leakage, first-time-right, on-time/on-budget, cost per release

Exit criteria. Both lanes run. The scorecard shows measured values and **blanks — not zeros — for anything a single run cannot evidence.**

What is deliberately not in this plan

Item	Why not
Agent role topology (a fixed specialist pool)	Collides with the downstream reuse decision; not answerable before Sprint 1 freezes <code>UserStory</code>
Persistent per-agent memory	Real value, but nothing in a one-week demo runs often enough to amortize it. Sprint 6+
A skill registry / plugin layer	The coverage model already carries the client-facing half of this. Build the registry only if a second consumer appears
Third-party agent skills or marketplace packages	Untrusted instructions in an agent's context, and an external dependency the locked-down sandbox will not have
Any benchmark figure not measured by this repo	Non-negotiable. See risk R3

Risks

#	Risk	Impact	Mitigation
R1	LLM access never approved	Sprint 3 cannot land; demo stays <code>STAGED</code>	Sprint 4 is decoupled by design. The staged labelling is honest and defensible on its own
R2	Locked-down sandbox will not serve a port	Console unusable	The CLI from Sprint 1 is the fallback. Confirm with the platform team by email now, not in the room
R3	A borrowed benchmark figure reaches a slide	Loses the room, and breaches another team's confidence	Only measured-here numbers ship. Empty states read "not measured", never <code>0</code>
R4	One-week clock; staffing unsettled	Sprints 4–5 slip	Sprints are ordered so slipping the tail still leaves a coherent demo. Sprint 2's recovery beat means a slip does not become a live failure
R5	Prompt prefixes restructured after recordings land	Every recording silently invalidated	Largely retired 2026-08-03: the ordering landed in Sprint 0, before any recording existed. Residual risk is editing <code>common/prompt.py</code> after Sprint 3 — <code>test_prompt_layers.py</code> pins the order so the change cannot be silent

Open items carried into the plan

- **Demo date and presentation format** — TBD. Sprint sizing is relative; the calendar is not yet set.
- **Staffing** — S7 needs more than one person; division of work unsettled.
- **Domain SME validation** — the disability submission scenario needs SME review of forms, required attachments, status names and pre-population rules.
- **LLM access** — see R1.
- **Browser availability in the locked-down environment** — see R2.